

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«АЛТАЙСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
Институт математики и информационных технологий
Кафедра информатики

**РАЗРАБОТКА БЭКЕНД-ЧАСТИ ДЛЯ ВЕБ-ПРИЛОЖЕНИЯ
ПРОКТОРИНГА**
(магистерская диссертация)

Выполнил:
студент 4.306М-3 группы
Пахомов Егор Иванович

(подпись)

Научный руководитель:
к.ф.-м.н., доцент
Козлов Денис Юрьевич

(подпись)

Допустить к защите:
Зав. кафедрой, к.ф.-м.н., доцент
Козлов Денис Юрьевич

(подпись)

«__» _____ 2025 г.

Работа защищена:
«__» _____ 2025 г.

Оценка: _____

Председатель ГЭК:

(подпись)

Барнаул 2025

РЕФЕРАТ

Тема выпускной квалификационной работы: «разработка бэкенд-части для веб-приложения прокторинга»

Цель работы – создание адаптируемого веб-приложения прокторинга с элементами искусственного интеллекта, обеспечивающего автоматизированный контроль за прохождением дистанционных экзаменов.

Предмет исследования – технологии и методы веб-разработки и искусственного интеллекта, применяемые в системах прокторинга.

Объект исследования – процессы дистанционного контроля знаний в системе высшего образования.

В результате исследования решены следующие задачи: проведен анализ существующих решений в области онлайн-прокторинга; сформулированы функциональные и технические требования к системе; разработана архитектура веб-приложения; реализован пользовательский интерфейс и серверная часть системы; внедрены базовые модули ИИ-анализа (распознавание лица, отслеживание взгляда, фиксация подозрительной активности); выполнено тестирование приложения в учебной среде; дана оценка его эффективности и возможности масштабирования в рамках образовательного учреждения.

Объем работы – 52 страницы печатного текста, количество рисунков – 7, таблиц – 1, приложений – 1, использованных источников литературы – 42.

Ключевые слова: прокторинг, искусственный интеллект, дистанционное обучение, контроль знаний, веб-разработка, компьютерное зрение, нейронные сети, идентификация пользователя.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ СИСТЕМ ПРОКТОРИНГА И ПРИМЕНЕНИЯ ИИ В ОБРАЗОВАНИИ	7
1.1 Актуальность дистанционного прокторинга и проблемы контроля знаний	7
1.2 Обзор отечественных систем прокторинга	8
1.3 Методы анализа поведения и компьютерного зрения	12
1.4 Роль искусственного интеллекта в образовательных системах	15
2 АНАЛИЗ ТРЕБОВАНИЙ И ПРОЕКТИРОВАНИЕ СИСТЕМЫ ПРОКТОРИНГА	18
2.1 Постановка задачи и целевая аудитория	18
2.2 Функциональные и нефункциональные требования	21
2.3 Архитектура приложения и выбор технологий.....	24
2.4. Проектирование базы данных.....	28
3 РЕАЛИЗАЦИЯ ВЕБ-ПРИЛОЖЕНИЯ ПРОКТОРИНГА.....	32
3.1 Структура реализации и архитектура проекта.....	32
3.2 Реализация REST API и логики обработки запросов.....	35
3.3 Интеграция с модулем анализа изображений	37
3.4 Хранение и отображение результатов прокторинга.....	40
3.5 Безопасность и контроль доступа.....	41
ЗАКЛЮЧЕНИЕ	44
БИБЛИОГРАФИЧЕСКИЙ СПИСОК	46
ПРИЛОЖЕНИЕ	51

ВВЕДЕНИЕ

В настоящее время в сфере образования активно развивается цифровизация учебных процессов, в том числе активно внедряются дистанционные и гибридные формы обучения. Этот процесс сопровождается необходимостью обеспечения объективного контроля знаний обучающихся и прозрачности проведения контроля знаний, что особенно важно при проведении экзаменов и итоговых работ. В связи с этим возрастает необходимость разработки систем, способных автоматизировать процесс наблюдений за студентами в ходе дистанционной проверки знаний – систем прокторинга.

Прокторинг представляет собой совокупность решений, предназначенных для контроля соблюдения правил сдачи онлайн тестирований и экзаменов. Одной из актуальных задач в данной области является разработка системы прокторинга с алгоритмами анализа видео потока, а также анализа поведения экзаменующихся. Важную роль в этом процессе играют технологии искусственного интеллекта, особенно компьютерное зрение и нейросетевые модели. Они позволяют отследить поведение пользователей и понять, происходило ли нарушение правил проверки знаний.

Актуальность настоящей работы обусловлена необходимостью создания адаптивного, масштабируемого и экономически эффективного веб-приложения прокторинга, интегрированного с элементами искусственного интеллекта, которое может быть внедрено в образовательный процесс университета. На сегодняшний день ряд коммерческих решений (таких как ProctorEdu, Examus, ProctorEdu и др.) предоставляют схожий функционал, однако они часто являются закрытыми, дорогостоящими и не всегда позволяют гибко адаптировать систему под специфические требования конкретного образовательного учреждения.

Целью магистерской диссертации является разработка серверной части веб-приложения прокторинга с элементами анализа поведения пользователя во время сдачи онлайн-экзамена, ориентированного на использование в учебной среде АлтГУ.

Для достижения поставленной цели необходимо решить следующие задачи:

- Выполнить обзор существующих решений в области прокторинга и анализа поведения;
- Определить требования к архитектуре и функциональности веб-приложения;
- Реализовать функциональность сбора и обработки видео данных с применением методов ИИ;
- Обеспечить базовый уровень анализа подозрительной активности с использованием алгоритмов компьютерного зрения.

Объект исследования — процессы организации и сопровождения дистанционного контроля знаний с использованием цифровых технологий.

Предмет исследования — архитектура и программная реализация веб-приложений прокторинга на основе методов искусственного интеллекта.

Методы исследования, применяемые в работе, включают анализ и синтез существующих технических решений, методологию системного проектирования, методы машинного обучения, компьютерного зрения, а также прототипирование и тестирование программных систем.

Информационная база исследования состоит из научных публикаций в области анализа поведения, документации по существующим системам прокторинга, материалов по веб-разработке, а также нормативных и

методических документов, регламентирующих проведение дистанционного обучения и контроля знаний в вузах РФ.

Научная новизна работы заключается в интеграции методов искусственного интеллекта в систему онлайн-прокторинга, разработанной специально под архитектуру и требования конкретного университета. Использование моделей компьютерного зрения для автоматического выявления подозрительного поведения обучающегося позволяет повысить эффективность и объективность контроля.

Практическая значимость работы заключается в возможности использования разработанного веб-приложения в реальной образовательной среде, а также в расширяемости системы под различные сценарии и учебные дисциплины. Кроме того, представленная система может послужить основой для дальнейших исследований и разработок в области мониторинга обучающихся.

Таким образом, разработка веб-приложения прокторинга представляет собой важную задачу на пересечении искусственного интеллекта, веб-технологий и образовательных информационных систем.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ СИСТЕМ ПРОКТОРИНГА И ПРИМЕНЕНИЯ ИИ В ОБРАЗОВАНИИ

1.1 Актуальность дистанционного прокторинга и проблемы контроля знаний

После 2020 года 67% вузов (данные Минобрнауки) ввели элементы дистанционного обучения, что сильно повлияло на весь процесс обучения. Появилась возможность обучаться не только полностью дистанционно, но и гибридно, то есть студенты могут приходить очно на некоторые предметы или отдельные темы. Сложность такого формата обучения в том, что студенты не взаимодействуют с преподавателями напрямую, преподаватели не могут полностью удостовериться, что студенты усвоили информацию, ведь проводя тестирования онлайн студент может достаточно легко жульничать. В таком формате проверки знаний студент может свободно пользоваться телефоном, переключаться между вкладками в браузере, ища ответы, получать подсказки извне, или же за него вовсе может сдавать тестирование другой человек [1]. Все это очень сложно отследить в дистанционном формате, даже если студенты и преподаватель находятся в одном звонке (сервисы видеосвязи). Не все студенты могут включить камеру, и отслеживать сразу 20-30 студентов в онлайн формате очень сложно. В связи с этими трудностями популярность набирают специальные системы по автоматическому отслеживанию действий студентов – прокторинговые системы.

Прокторинг позволяет автоматически или полуавтоматически отслеживать нарушения, фиксировать их и предоставлять отчет о зафиксированных нарушениях [2]. Такие системы сильно упрощают задачу отслеживания поведения студентов во время дистанционных тестирований/экзаменов. Существует несколько подходов к организации прокторинга:

- Синхронный прокторинг – контроль в режиме реального времени, часто с участием оператора;
- Асинхронный – запись и последующий анализ действий;
- Автоматический прокторинг – основан полностью на компьютерном зрении и анализе действий студента;
- Гибридный подход – совмещение первого и второго способов.

Преимущества прокторинга очевидны, он позволяет одновременно отслеживать большое количество студентов, им достаточно удобно пользоваться, в случае с автоматически прокторингом не требуется задействования преподавателей, что освобождает их время на выполнение других задач. Однако такие системы сложны в реализации, требуют точных алгоритмов, соблюдения требований безопасности (защищенная передача данных), соблюдение конфиденциальности студентов [3]. Такие способы отслеживания действий студентов сложны не только в технической реализации, но и вызывают этические вопросы.

1.2 Обзор отечественных систем прокторинга

В настоящее время системы прокторинга набирают популярность, но не все из них подходят для использования. Все большую популярность набирают отечественные системы прокторинга, они соответствуют законодательству и можно не переживать, что данные будут утекать и использоваться в корыстных целях. Такие системы имеют интерфейс на русском языке, есть возможность связаться с разработчиками этих систем, а также некоторые из них имеют возможность интегрироваться с LMS. Рассмотрим более подробно некоторые из этих систем.

Examus – одна из наиболее известных российских платформ прокторинга [4]. Она использует методы компьютерного зрения для

отслеживания поведения обучающегося во время сдачи экзаменов. Система анализирует повороты головы, направление взгляда, мимику, наличие посторонних лиц и звуков. Отчёты Examus формируются автоматически, а также могут быть проверены человеком-проктором. Решение поддерживает интеграцию с LMS, такими как Moodle и Open edX, и широко применяется в университетах, включая МГУ, НИУ ВШЭ, УрФУ и другие.

РЭУ Про – система прокторинга, разработанная при участии Российского экономического университета имени Г.В. Плеханова [5]. Она ориентирована на вузовскую среду и предоставляет функции видеофиксации экзамена, контроля за окнами браузера, записи экрана и аудиоканала. Система позволяет преподавателю проверять сессии вручную или полагаться на автоматическую маркировку подозрительных действий.

ProctorEdu – российская облачная система прокторинга, ориентированная на интеграцию с образовательными платформами [6]. Поддерживает автоматическую идентификацию личности, фиксацию подозрительных действий, анализ поведения, ведёт видео- и аудиозапись с последующей обработкой ИИ. Особенностью является работа даже при нестабильном интернете – видео сохраняется локально и загружается после завершения сессии.

Прокторинг от Stepik и МФТИ – решение, созданное для проведения массовых открытых онлайн-курсов (МООК) с контролем знаний. Используется в том числе для прокторинга на олимпиадах и сертификационных экзаменах. Позволяет отслеживать лицо пользователя, блокировать переключение вкладок, записывать видео- и аудиопоток. Интегрируется с платформой Stepik [7, 8].

Общие функциональные возможности российских систем:

- Распознавание и верификация личности (лицевая биометрия);
- Отслеживание направления взгляда, движения головы;
- Запись видео и аудио во время экзамена;
- Детекция посторонних лиц или голосов;
- Автоматическое формирование отчётов для преподавателя;
- Интеграция с LMS и СДО (Moodle, Open edX и др.).

Преимущества отечественных решений:

- Соответствие законодательству РФ (включая 152-ФЗ "О персональных данных");
- Поддержка русскоязычного интерфейса;
- Возможность размещения на локальных серверах ВУЗа;
- Интеграция с внутренними ИС и электронными журналами.

Однако остаются и проблемы:

- Высокая стоимость лицензий некоторых коммерческих решений;
- Недостаточная точность автоматических алгоритмов в условиях плохого освещения или нестандартного оборудования;
- Недоверие со стороны студентов из-за приватности и непрозрачности работы систем [9].

Таблица 1.1

Сравнительная таблица систем прокторинга

Система	ИИ и CV технологии	Основные функции	Тип прокторинга	Интеграция с LMS	Особенности и применение
Examus	Распознавание лиц, трекинг головы и взгляда (OpenCV, ML)	Видеоаналитика, анализ поведения, фиксация событий, отчёты	Автоматический / гибридный	Moodle, Open edX	Используется в МГУ, ВШЭ, интеграция с СДО
ProctorEdu	Распознавание лиц, анализ поведения (AI)	Идентификация, видеозапись, анализ событий, автономная работа	Автоматический	API, Open edX	Работает при нестабильном интернете, SaaS
РЭУ Про	Распознавание лиц, контроль вкладок, звук	Запись видео/аудио, блокировка переключения окон, отчёты	Гибридный	Прототип LMS РЭУ	Ориентирована на вузовские нужды
Stepik + МФТИ	Детекция лиц, запись поведения	Запись экрана/видео, блокировка вкладок, простая идентификация	Асинхронный	Stepik	Используется на олимпиадах, MOOK, экзаменах МФТИ

Таким образом, в России уже сформировался рынок решений прокторинга, адаптированных под нужды образовательных организаций. Тем не менее, остаётся запрос на более гибкие, масштабируемые и доступные по цене системы, которые могли бы быть доработаны или разработаны вузами самостоятельно. Это определяет актуальность создания веб-приложения прокторинга с открытой архитектурой, расширяемой функциональностью и поддержкой современных ИИ-технологий.

1.3 Методы анализа поведения и компьютерного зрения

Системы прокторинга все чаще используют технологии компьютерного зрения для анализа и отслеживания поведения пользователя, при проведении дистанционных тестирований и экзаменов [10]. Основная задача этих технологий – объективно фиксировать действия пользователей, обеспечить выявление нарушений во время прохождения тестирований. Такие технологии в основном используются при проведении автоматического прокторинга. Далее будут рассмотрены технологии, которые используются в системах автоматического прокторинга.

Распознавание лица – основная функция, которая используется как в самом начале, при идентификации пользователя, так и на протяжении всей сессии прокторинга. Она позволяет отследить наличие человека в кадре, проверить тот ли человек проходит тестирование и нет ли посторонних людей в кадре.

На практике используются:

- Библиотека face-recognition на базе dlib, обеспечивающая точное выделение лицевых признаков [11, 12];
- Алгоритмы гистограмм направленных градиентов (HOG) и сверточные нейронные сети (CNN) [13, 14];

- Модели, обученные на датасетах LFW, VGGFace, MS-Celeb и других [15, 16, 17].

При валидации пользователя система сопоставляет изображение, полученное с камеры пользователя с эталонным изображением, которое было загружено в систему заранее. Это может происходить единожды, в начале сессии прокторинга, или на протяжении всей сессии прокторинга. Пользователь может незначительно отличаться от эталонной фотографии, поэтому устанавливается порог допустимости, который указывает на сколько пользователь может отличаться от эталонной фотографии. В случае отклонения, превышающего этот порог, система указывает на возможную подмену личности.

Отслеживание взгляда и поворота головы позволяет понять, что пользователь использует посторонние материалы в качестве подсказок. Он может отвести взгляд в сторону, чтобы посмотреть на заготовленные ответы, воспользоваться телефоном или посоветоваться с кем-то, кого не видно в кадре [18].

Технологии:

- MediaPipe Face Mesh – библиотека от Google, позволяющая получать до 468 ключевых точек лица в реальном времени [19];
- Определение координат зрачков, век и углов наклона головы;
- Вычисление вектора взгляда и отклонения от направления на экран.

Если обучающийся отводит взгляд вниз или в сторону, система может засчитать это как попытку жульничества и зафиксировать нарушение. Также система позволит понять куда направлена голова пользователя и меняет ли он ее положение.

Также частым нарушением является присутствие третьих лиц при тестировании. Посторонние могут подсказывать студенту, что не позволяет оценить настоящие знания обучающегося. Для обнаружения посторонних используют алгоритмы обнаружения нескольких лиц. Если на кадре обнаруживается больше одного лица, система фиксирует это нарушение.

Методы:

- Haar Cascade, SSD, YOLO – для обнаружения лиц на изображении [20, 21, 22];
- Многокадровый анализ – чтобы избежать ложных срабатываний при появлении теней, отражений или предметов;
- Анализ аудиопотока – выявление нескольких голосов, громкой речи, характерного фона (например, шёпота, команд, шума от листания бумаги).

Еще один способ получить подсказки – использовать компьютер, на котором происходит тестирование, с него можно выйти в интернет, открыть сторонние файлы и тд. Для фиксации таких действий студентов используются специальные алгоритмы отслеживания поведения пользователя с использованием интерфейсных данных – движение мыши, переключение между окнами, открытие дополнительных вкладок и тд [4]. При обнаружении посторонних действий система фиксирует нарушение.

Для этого используются:

- Модули записи экрана;
- Контроль процессов на уровне системы (в десктопных клиентах);
- Интеграция с безопасными браузерами или расширениями.

По завершении сессии система формирует отчёт, в котором фиксируются:

- События (например, "обнаружено отклонение головы");
- Скриншоты и фрагменты видео с временными метками;
- Уровень подозрительности, рассчитанный на основе набора эвристик или модели машинного обучения.

Алгоритмы могут быть как эвристическими, так и обученные на размеченной выборке функций с помощью методов классификации и ранжирования.

1.4 Роль искусственного интеллекта в образовательных системах

В настоящее время искусственный интеллект активно внедряется в образовательную систему, влияя как на процессы обучения и преподавания, так и на процесс контроля знаний. Искусственный интеллект задействован в сборе, обработке и интерпретации больших массивов данных. Обработывая данные об активности студентов, посещаемости, прохождении тестирований, курсов, выполнении заданий, он позволяет прогнозировать успеваемость студентов, выявлять отстающих, предлагать индивидуальные рекомендации и тд [23]. Помимо помощи студентам и преподавателям искусственный интеллект становится важной частью прокторинговых систем, позволяя проводить прокторинг без участия человека, обеспечивая высокоточный контроль и достаточно простую масштабируемость.

Примеры использования ИИ в образовательных учреждениях:

- Адаптивные обучающие системы (например, Яндекс.Учебник, SmartLMS);
- Аналитические модули в LMS, использующие машинное обучение для предсказания вероятности успешной сдачи экзамена.

В системах прокторинга ИИ применяется для:

- Автоматического распознавания лиц, анализа мимики, направления взгляда;
- Детекции аномалий в поведении на основе обученных моделей (например, резкие движения, частые отклонения от экрана, посторонние звуки);
- Классификации экзаменационных сессий по степени подозрительности;
- Построения отчётов с оценкой риска нарушения.

Основным преимуществом использования искусственного интеллекта является способность обнаружить скрытые закономерности и признаки в поведении студентов, которые указывают на нарушения и возможно не всегда будут замечены человеком проктором.

Несмотря на преимущества, использование искусственного интеллекта вызывает ряд этических вопросов:

- Конфиденциальность данных – системы прокторинга обрабатывают чувствительную биометрическую информацию, включая изображение лица, голос, окружение;
- Информированное согласие – студенты должны быть заранее уведомлены о механизмах контроля и дать явное согласие на обработку персональных данных;
- Прозрачность алгоритмов – алгоритмы ИИ часто являются "чёрным ящиком", что затрудняет объяснение оснований для вынесенных решений;
- Недоверие и стресс у студентов – наличие постоянного наблюдения может вызывать повышенное беспокойство и ощущение несправедливости.

В условиях российской юрисдикции особенно важно соблюдать положения Федерального закона №152-ФЗ "О персональных данных" [24], а также учитывать международные практики, такие как принципы GDPR [25] (если система работает с иностранными студентами). GDPR – это общий регламент по защите данных Европейского союза, который устанавливает правила обработки персональных данных гражданами ЕС.

Интеграция ИИ в образовательные платформы будет только усиливаться. Ожидаются дальнейшие улучшения в области:

- Распознавания эмоций и мимики как индикатора вовлечённости;
- Индивидуализации контроля и обучения;
- Самообучающихся систем, адаптирующих параметры прокторинга под тип экзамена и профиль обучающегося;
- Объединения прокторинга с обучающей аналитикой и цифровым портфолио студента.

2 АНАЛИЗ ТРЕБОВАНИЙ И ПРОЕКТИРОВАНИЕ СИСТЕМЫ ПРОКТОРИНГА

2.1 Постановка задачи и целевая аудитория

В последние годы онлайн-обучение стало нормой, многие вузы переходят на дистанционный или гибридный формат проведения занятий, это способствует развитию образовательных технологий. Преподавание теоретического материала и выполнение заданий дистанционно были успешно адаптированы, для этого разработано большое количество программ, но многие образовательные учреждения используют системы собственной разработки. Проблемой стал объективный контроль знаний студентов, отсутствие инструментов для подтверждения личности студентов и наблюдения за процессом сдачи экзамена, что значительно снижает достоверность результатов [26].

Проблема становится сложнее из-за того, что традиционные формы контроля (очные экзамены, наблюдение проктором) теряют свою применимость в онлайн-среде. Обычное тестирование в системе дистанционного обучения не гарантирует честность студентов, они могут пользоваться сторонней помощью, телефонами, шпаргалками, конспектами или готовыми ответами. Также студентам могут помогать третьи лица или вовсе проходить тестирование за студента. Исходя из всех этих факторов возникает необходимость в разработке системы прокторинга, которая будет интегрирована в существующую образовательную систему ВУЗа. Это позволит обеспечить дистанционный контроль за студентами с использованием методов компьютерного зрения и анализа поведения студентов.

Цель разрабатываемой системы – создание прототипа веб-приложения для автоматизированного прокторинга, способного выполнять:

- идентификацию личности обучающегося на основе изображения с веб-камеры;
- мониторинг активности пользователя во время прохождения контроля знаний;
- регистрацию потенциально подозрительных действий;
- формирование отчёта по результатам сессии.

Разрабатываемое программное обеспечение ориентировано на преподавателей и учебно-методических специалистов высших учебных заведений. Именно они будут использовать систему для:

- назначения экзаменационных сессий;
- выбора типа прокторинга (например, распознавание лица, анализ взгляда, фиксация событий);
- просмотра и анализа отчётов;
- интеграции с электронными журналами и платформами дистанционного обучения.

Студенты также будут пользоваться платформой, но они будут только проходить прокторинг, результаты они не увидят, повлиять на них не смогут. Однако основным пользователем остается преподаватель и администратор системы.

Для более наглядного представления взаимодействия пользователей с системой прокторинга была построена диаграмма вариантов использования, представленная на рисунке 2.1. Она отражает основные роли — студента, преподавателя и администратора — и доступные им действия в рамках платформы.

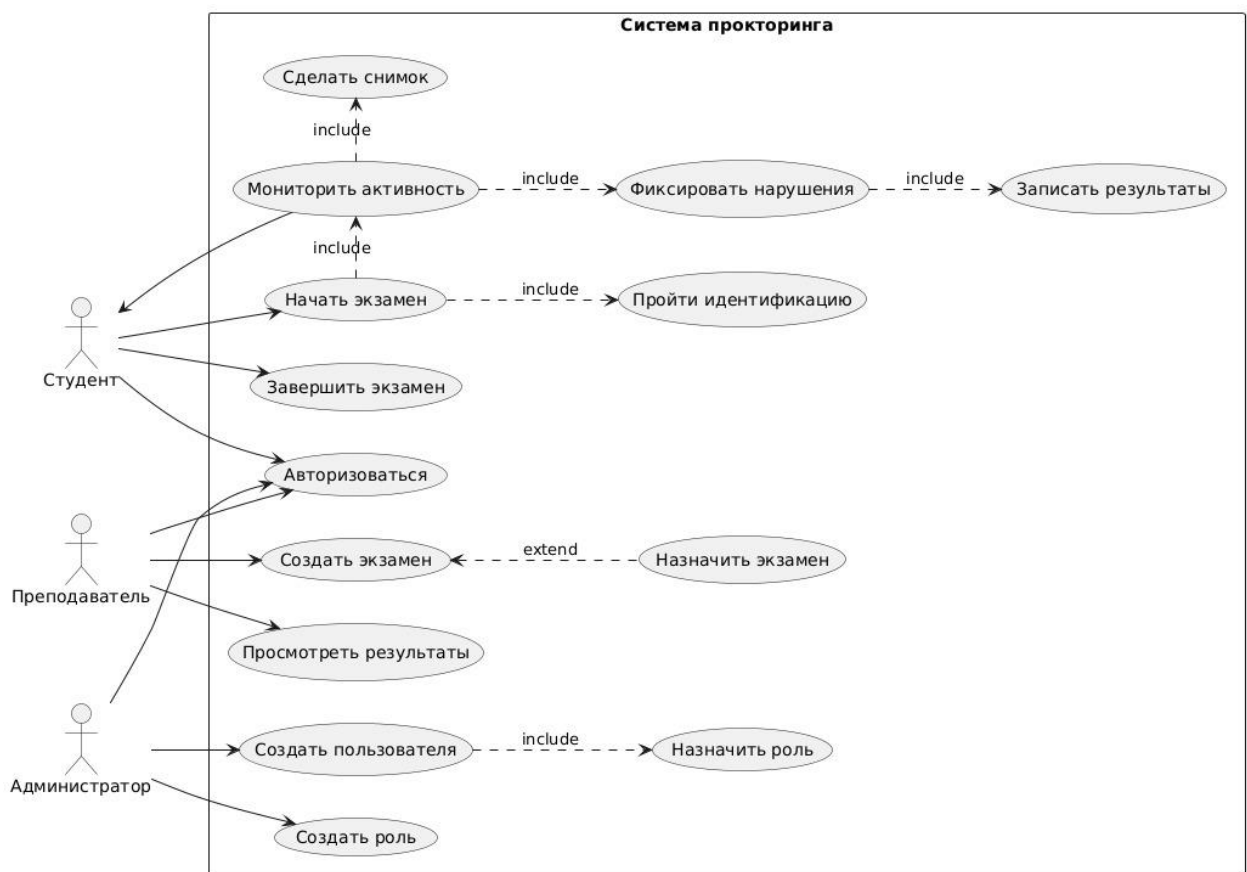


Рисунок 2.1 – Use Case диаграмма использования системы прокторинга

Преподаватели могут создавать сессии прокторинга, типы прокторинга, предметы для прокторинга, назначать сессии прокторинга на определенных студентов и анализировать результаты. Студенты проходят авторизацию, идентификацию и проходят сессию прокторинга во время сдачи экзамена. Администраторы управляют пользователями и их ролями.

При проектировании и разработке учитываются следующие ограничения:

- Работа в среде ограниченных ресурсов (например, не все студенты используют современные устройства или стабильный интернет);
- Необходимость соблюдения законодательства в области персональных данных;

- Простота интерфейса для преподавателей без технической подготовки;
- Возможность масштабирования и интеграции с существующей ИТ-инфраструктурой вуза.

Таким образом, постановка задачи сводится к созданию надёжного и доступного веб-приложения, ориентированного на преподавателей, позволяющего эффективно организовать и контролировать процесс дистанционного тестирования с минимальными усилиями и максимальной объективностью.

2.2 Функциональные и нефункциональные требования

Этап проектирования любой системы – очень важный этап, во время которого четко формируются требования, которым система должна соответствовать. Также во время проектирования системы можно понять, какие сложности возникнут при разработке, можно сформировать техническое задание и обеспечить целенаправленную разработку архитектуры и функционала.

Разрабатываемое приложение – это REST API [27], предоставляющее функционал администраторам платформы, преподавателям и студентам ВУЗа. Основными требованиями стали надежность, безопасность, удобство использования и контроль нарушений.

Функциональные требования

1. Авторизация и аутентификация пользователей

- Регистрация и вход по роли (администратор / преподаватель);
- Защита паролей (хеширование, токены доступа);

- Разграничение прав доступа.
2. Управление предметами и типами прокторинга
- Создание, редактирование и удаление дисциплин;
 - Привязка преподавателей к предметам;
 - Назначение типа прокторинга (например: фиксация лица, трекинг взгляда, базовый режим без анализа).
3. Создание и управление сессиями прокторинга
- Назначение экзамена / контрольной;
 - Генерация уникальной сессии с временными рамками;
 - Отправка ссылки участникам (в перспективе – автоматическая привязка к LMS).
4. Распознавание и анализ
- Идентификация лица при входе в сессию;
 - Непрерывное отслеживание ключевых параметров: наличие лица, направление взгляда, посторонние объекты;
 - Фиксация времени начала и завершения сессии.
5. Отчётность
- Просмотр сессии преподавателем;
 - Список зафиксированных событий;
6. Управление пользователями
- Создание и деактивация аккаунтов преподавателей (администратором);
 - Назначение ролей и дисциплин.

Нефункциональные требования

1. Производительность

- Обработка запросов REST API в течение ≤ 500 мс при стандартной нагрузке;
- Поддержка одновременной работы ≥ 20 сессий (в рамках прототипа).

2. Масштабируемость

- Возможность развёртывания в Docker-контейнере [28];
- Перспектива разделения логики по микросервисам (в будущем).

3. Надёжность

- Ведение логов ошибок;
- Устойчивость к некорректным данным;
- Проверка валидности запросов.

4. Безопасность

- Хеширование паролей (например, bcrypt);
- JWT-авторизация [29];
- Защита от SQL-инъекций и CSRF-атак.

5. Юзабилити

- REST API с открытой документацией (Swagger/OpenAPI [30]);
- Простой и однозначный формат запросов и ответов;
- Возможность интеграции с внешними сервисами.

6. Соблюдение законодательства

- Хранение и обработка персональных данных в соответствии с ФЗ-152;
- Возможность локального развертывания системы на серверах вуза.

Определенные функциональные и нефункциональные требования служат основой для дальнейшей проработки архитектуры и реализации системы. Они также позволяют определить критерии для тестирования системы и оценки качества работы приложения.

2.3 Архитектура приложения и выбор технологий

Разрабатываемая система прокторинга – это серверная часть веб-приложения с REST API архитектурой, с которым могут взаимодействовать внешние клиенты (клиентская часть приложения, LMS или внешние системы). В основе лежит монолитная архитектура, что обосновано прототипным характером проекта, упрощённой схемой развёртывания и ограниченным числом пользователей на этапе тестирования. Приложение построено по принципу "клиент-сервер", где клиентская часть (в перспективе – отдельный фронтенд или LMS) обращается к REST API, реализованному на сервере. Компоненты серверной архитектуры:

- FastAPI – основной веб-фреймворк, обеспечивающий маршрутизацию запросов, валидацию данных, генерацию OpenAPI-документации и высокую производительность [31];
- PostgreSQL – реляционная база данных, выбранная за надёжность, масштабируемость и поддержку расширенных типов данных [32];

- SQLAlchemy – ORM для работы с базой данных, реализующая паттерн "единица работы" и позволяющая отделить бизнес-логику от SQL-запросов [33];
- Alembic – система миграций базы данных, тесно интегрированная с SQLAlchemy [34];
- Pydantic – библиотека для декларативной валидации входных и выходных моделей API [35];
- JWT (JSON Web Token) – для аутентификации и авторизации пользователей с разграничением ролей.

Приложение реализовано по принципу многослойной архитектуры, что позволяет чётко разделить ответственность между компонентами [36]:

- Слой представления (роутеры) – обработка HTTP-запросов [37];
- Слой бизнес-логики (сервисы) – обработка сценариев;
- Слой доступа к данным (DB-сервисы) – SQLAlchemy-запросы;
- Слой ИИ-анализа – обработка видео сторонним модулем mediapipe [38].

Упрощенная схема многослойной архитектуры приложения представлена на рисунке 2.2.

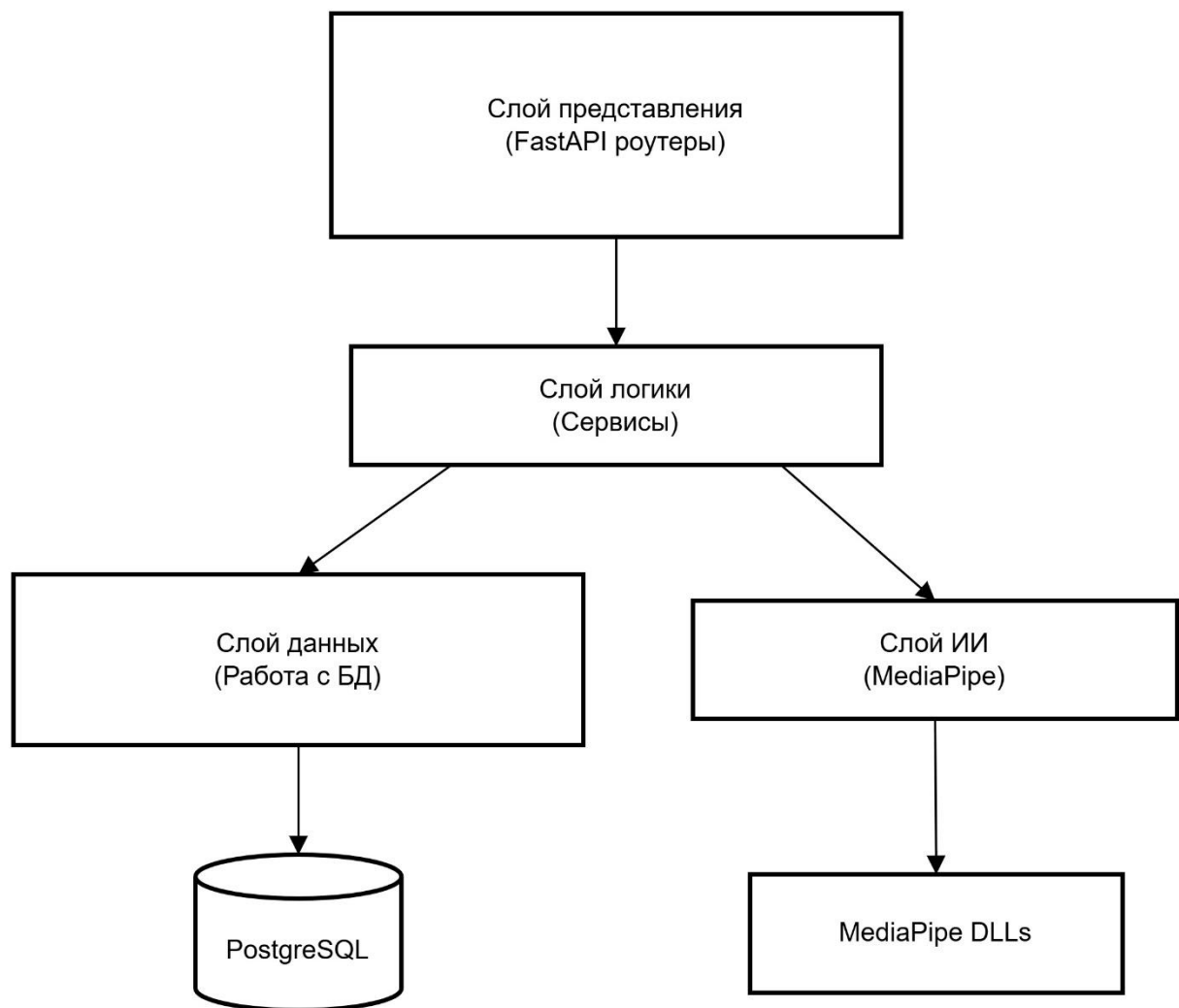


Рисунок 2.2 – Многослойная архитектура веб-приложения прокторинга

Для анализа изображения с веб-камеры используются следующие библиотеки:

- face-recognition – библиотека на основе dlib, обеспечивающая детекцию и верификацию лица обучающегося. Используется на этапе входа в сессию и в процессе её прохождения;
- MediaPipe – библиотека от Google, позволяющая отслеживать взгляд, положение головы и наличие лица в кадре в реальном времени. Подходит для онлайн-трекинга и выявления отклонений от экрана;

- OpenCV – универсальная библиотека для работы с изображениями и видеопотоком [39]. Применяется для чтения/записи видео, базовой обработки кадров (поворот, масштаб, фильтрация) и передачи данных в другие модули.

Структура проекта построена по модульному принципу, что упрощает поддержку и масштабирование. Она представлена на рисунке 2.3.

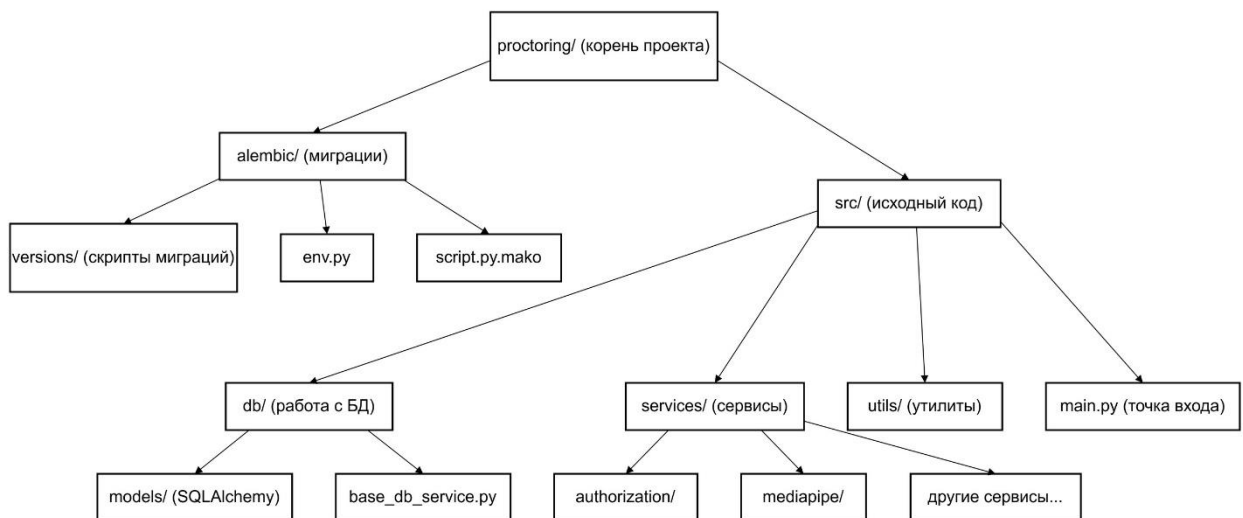


Рисунок 2.3 – Структура каталогов проекта

На логическом уровне серверное приложение можно разделить на следующие модули:

1. Модуль пользователей

- Регистрация, аутентификация, роли
- Управление аккаунтами преподавателей и администраторов

2. Модуль дисциплин и сессий

- Создание и настройка экзаменов
- Назначение прокторинга и сроков
- Привязка к преподавателям

3. Модуль прокторинга

- Детектирование лица и анализ поведения
- Обработка видеопотока с веб-камеры
- Фиксация событий: исчезновение лица, отклонение взгляда, появление второго лица

4. Модуль хранения и отчётности

- Сохранение результатов сессии
- Формирование отчёта
- Логирование и отладочная информация

Монолитная архитектура выбрана по следующим причинам:

- Простота разработки и отладки на этапе прототипа;
- Отсутствие необходимости в разделении нагрузки на микросервисы;
- Возможность локального развертывания для ручного тестирования.

В будущем, при расширении системы, возможен переход к микросервисной архитектуре с выносом модулей прокторинга, аналитики и хранения в отдельные сервисы.

2.4. Проектирование базы данных

База данных является основой хранения всей информации системы. В не хранятся данные о пользователях, ролях, предметах, сессиях и типах прокторинга. Проектирование структуры базы данных выполнялось вручную на основе анализа требований к системе, а также учитывая нормальные формы [40]. В качестве диалекта был выбран PostgreSQL.

В базе данных реализованы следующие ключевые сущности:

- Таблица `user` – хранит информацию о пользователях, включая их идентификаторы и учетные данные;
- Таблица `role` – определяет роли пользователей (например, преподаватель, администратор) и их права;
- Таблица `subject` – содержит информацию о предметах, связанных с экзаменами;
- Таблица `authorization` – хранит данные авторизации пользователей;
- Таблица `proctoring` – хранит данные о прокторинге, связывая экзамены с пользователями и предметами;
- Таблица `proctoring_type` – описывает типы прокторинга, применяемые к экзаменам;
- Таблица `proctoring_result` – описывает результаты прокторинга, применяемые к экзаменам.

Модель базы данных была выстроена с ориентацией на третью нормальную форму, что позволило исключить дублирование информации, реализовать четкие связи между сущностями и упростить работу с данными. Важной частью базы данных является таблица пользователей, в которой хранится основная информация о пользователях, в том числе логин, пароль, электронная почта и привязка к определенной роли. Система ролей реализована в виде отдельной таблицы с указанием прав доступа пользователей.

Большое внимание было уделено связям между таблицами пользователей и предметов. Поскольку в таблице пользователей хранятся данные о преподавателях, администраторах и студентах необходимо было сделать возможным назначения нескольких предметов одному пользователю и наоборот, несколько пользователей на один предмет. Например,

тестирование по предмету будут проходить несколько студентов, значит необходимо их привязать к одному предмету, с другой стороны, необходимо предоставить преподавателю возможность просматривать результаты прокторинга по нескольким предметам. Это классическая ситуация в построение связей таблиц, для решения этой проблемы используется связь многие ко многим. Для реализации этой связи необходимо реализовать вспомогательную таблицу, в которой будет храниться идентификаторы пользователей и предметов.

Сама сессия прокторинга представлена в базе данных как отдельная сущность, она объединяет в себе информацию о студенте, который проходит прокторинг, предмете, типе прокторинга и тд. Тип прокторинга также вынесен в отдельную сущность, в ней хранится информация о том, что необходимо отслеживать во время прохождения тестирования, например, детекция лица, анализ направления взгляда, фиксация посторонних звуков, определение наличия посторонних лиц и тд.

Результаты прокторинга вынесены в отдельную таблицу связанную с определенной сессией прокторинга. Такая декомпозиция обоснована тем, что на одну сессию прокторинга может приходиться большое кол-во зафиксированных нарушений, что требует агрегации информации. Таблица результатов похожа на таблицу типов прокторинга, поля таблицы содержат булевы значения, которые отвечают за обнаружение нарушений во время прохождения тестирований.

На этапе реализации структуры БД использовалась система управления базами данных PostgreSQL, обеспечивающая надёжность, масштабируемость и соответствие требованиям ACID [41]. ACID – это набор свойств транзакций в базах данных, гарантирующих надёжность операций: Атомарность (Atomicity, все изменения выполняются целиком или не выполняются вовсе), Согласованность (Consistency, данные всегда остаются корректными),

Изолированность (Isolation, параллельные транзакции не мешают друг другу) и Долговечность (Durability, результаты сохраняются даже после сбоев). Взаимодействие с базой данных будет реализовано через ORM SQLAlchemy – объектно-реляционное отображение на Python, которое позволяет работать с моделями базы данных как с классами. Для управления актуальностью базы данных и контроля ее версий будет использована библиотека Alembic, она отлично работает с выбранной ORM. Alembic автоматически сверяет текущее состояние базы данных с моделями, описанными в ORM, находя различия он создает миграцию. Благодаря этим миграциям мы можем вносить изменения в базу данных и быстро откатиться в случае возникновения проблем.

На рисунке 2.4 приведена схема базы данных, отображающая основные сущности, их поля и связи. Она отражает концептуальную модель системы и служит основой для взаимодействия всех компонентов веб-приложения с хранилищем данных.

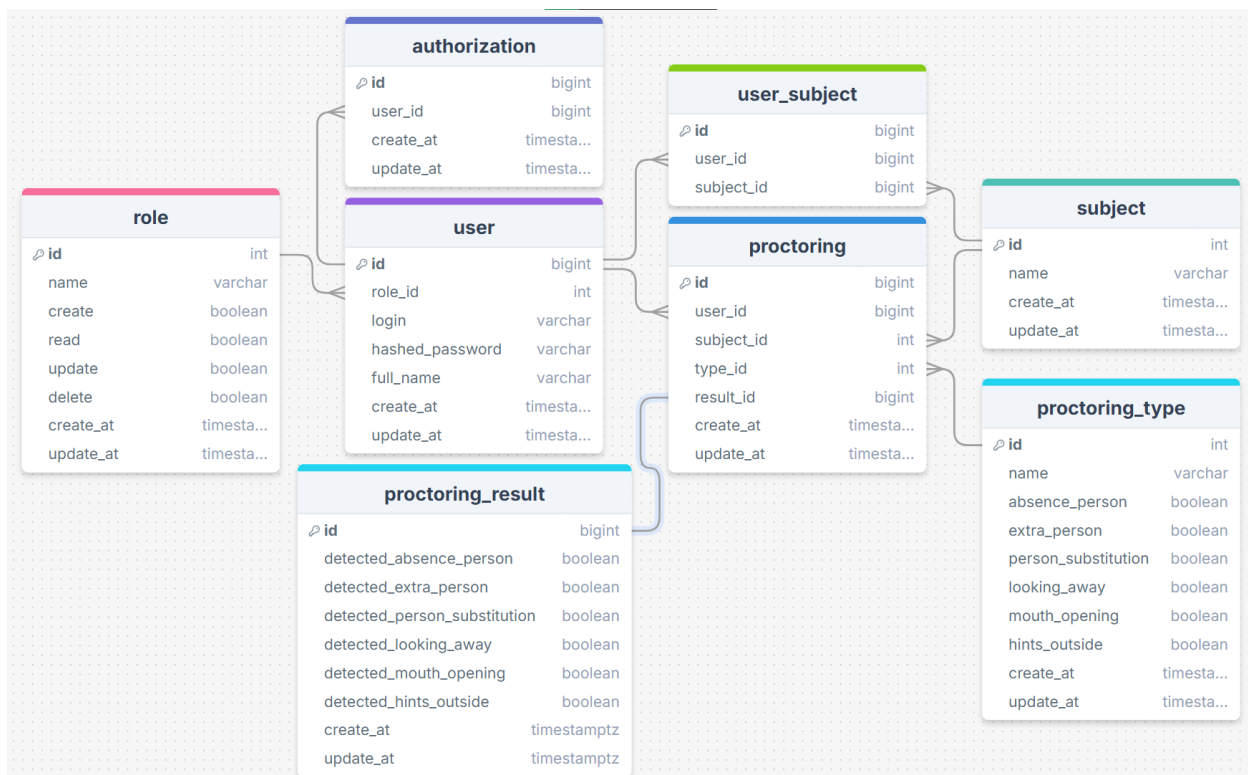


Рисунок 2.4 – Схема базы данных веб-приложения прокторинга

3 РЕАЛИЗАЦИЯ ВЕБ-ПРИЛОЖЕНИЯ ПРОКТОРИНГА

3.1 Структура реализации и архитектура проекта

Разработка серверной части приложения происходила при помощи языка Python, в качестве основного фреймворка был выбран FastAPI. Как было описано в главе 2, была выбрана многослойная архитектура приложения, которую успешно получилось реализовать. В таком подходе каждая часть приложения разделяется по уровням ответственности. Таким образом получается следующая архитектура: маршруты (router), сервисы с бизнес логикой (service), сервисы взаимодействия с базой данных (db_service), схемы данных (schemas), исключения (exceptions) и зависимости модуля (dependeceis).

Весь исходный код приложения расположен в каталоге `src/`, внутри которого содержатся ключевые модули:

- `services/` – включает модули бизнес-логики, такие как `user`, `authorization`, `proctoring`, `proctoring_result`, каждый из которых реализует отдельный сервис;
- `services/proctoring/routers/` – содержит маршруты API, сгруппированные по версиям и логике (например, `router_v1.py`);
- `db/models/` – описывает модели SQLAlchemy, соответствующие таблицам БД;
- `utils/` – включает вспомогательные функции, например, криптографию и проверку ролей;
- `main.py` – точка входа в приложение, где происходит инициализация FastAPI и подключение всех маршрутов;
- `config.py` – модуль конфигурации, где определяются параметры подключения к БД, JWT и другие переменные среды.

HTTP запросы, поступающие в приложение от клиентов, обрабатываются маршрутами FastAPI, где происходит проверка токена пользователя через зависимость Depends. Далее запрос попадает в соответствующий сервис с бизнес-логикой, методы обернуты в декораторы, которые проверяют соответствующие права пользователей. При необходимости запрос передается в сервис базы данных и исходя из типа запроса, происходит взаимодействие с базой данных.

Ниже, на рисунке 3.1, представлена структура каталогов приложения.

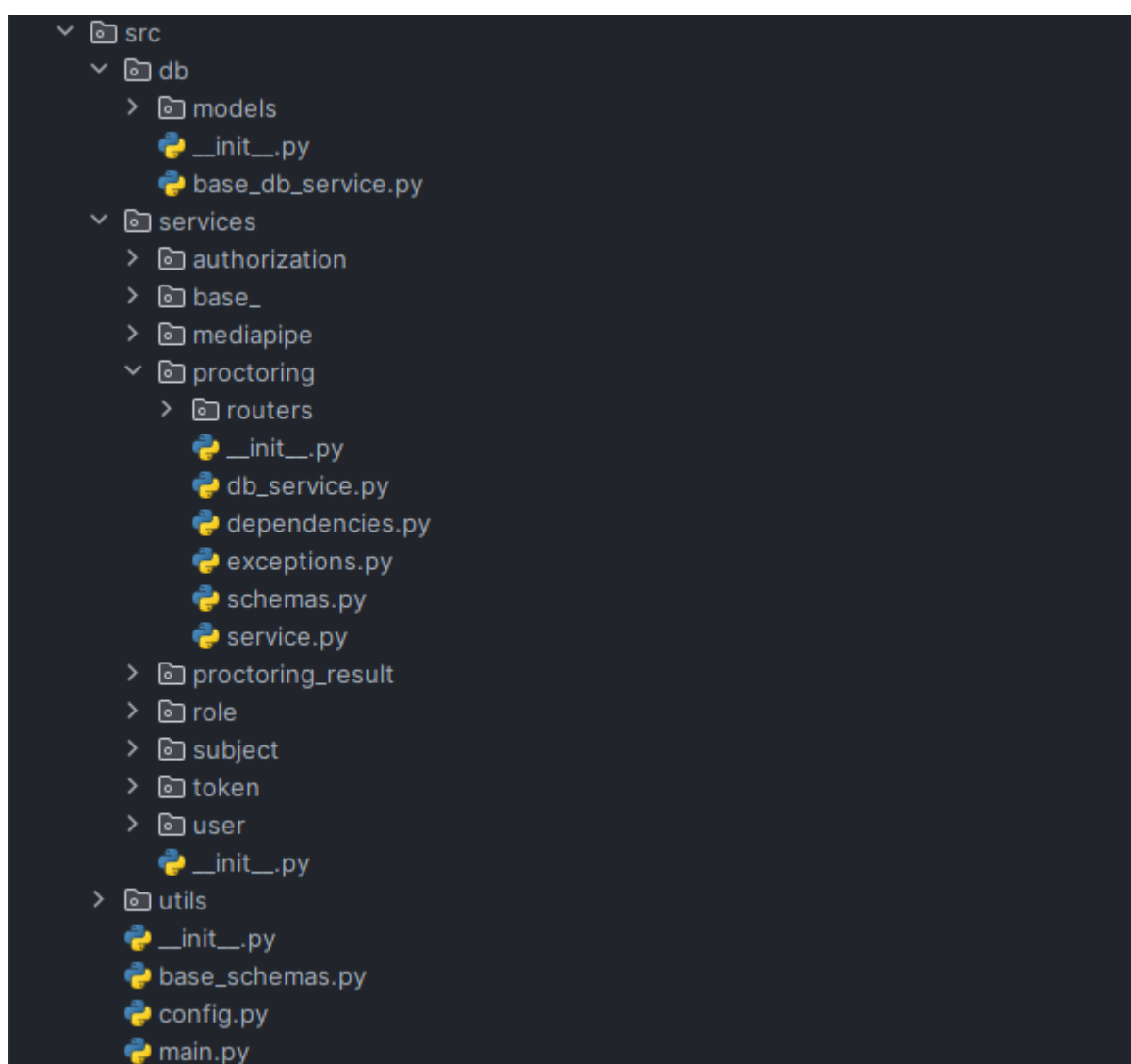


Рисунок 3.1 – Структура исходного кода серверной части веб-приложения

Также была реализована поддержка миграций базы данных, через Alembic, что позволяет контролировать состояние базы данных через различные ревизии. Скрипты миграций расположены в корневом каталоге в папке alembic/versions/. Каждый скрипт –это отдельная миграция, у каждой миграции есть своя ревизия (идентификатор), в нем прописываются все операции по изменению базы данных.

В итоге у нас получается модульная организация приложения, которая позволила оформить каждую бизнес-сущность в виде отдельного модуля с собственными роутерами, схемами, сервисами и исключениями. Получившаяся структура приложения представлена на рисунке 3.2. Благодаря такому подходу систему можно очень просто и быстро расширять.

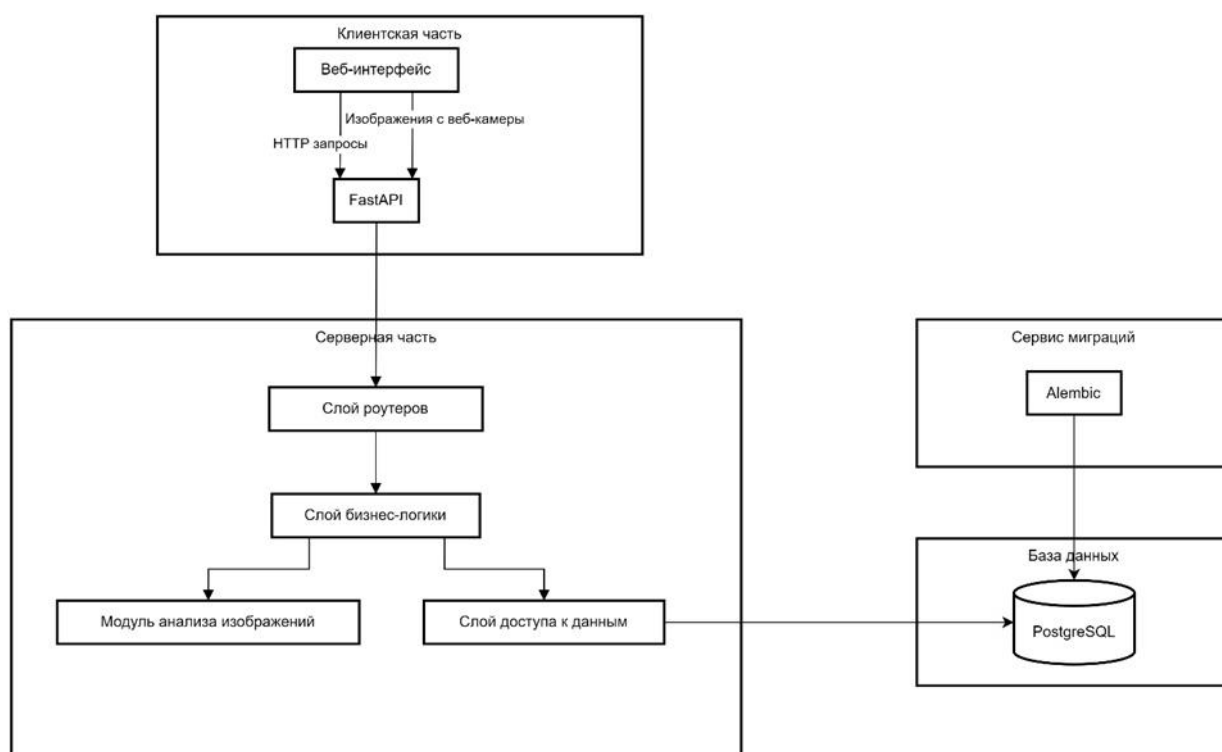


Рисунок 3.2 – Структура приложения

3.2 Реализация REST API и логики обработки запросов

REST-интерфейс является ключевой точкой взаимодействия приложения с клиентами. Он построен на современном веб-фреймворке FastAPI, который обеспечивает декларативное определение маршрутов, автоматически собирающуюся документацию, строгую типизацию входных и выходных данных (при использовании Pydantic), простое подключение зависимостей и поддержку асинхронности.

Каждый модуль отвечает за свою логическую часть, и находится в директории `services`, в каждом сервисе есть папка с маршрутами, `routers`. В файле `main.py` происходит регистрация всех маршрутов из всех сервисов. Каждый маршрут оформлен как функция с аннотациями типов и зависимостями. Взаимодействие с бизнес-логикой осуществляется через сервисы, внедряемые с помощью механизма `Depends`.

Ниже приведён пример реализации маршрута получения информации о прокторинговой сессии:

```
@router.get("/{proctoringId}",
status_code=status.HTTP_200_OK)
async def get_proctoring_by_id(
    proctoring_id: int = Path(alias="proctoringId"),
    proctoring_service: ProctoringService = (
        Depends(proctoring_service_dependency)
    ),
    user: User = Depends(decode_user_token),
) -> ProctoringItemSchema:
    return await (
        proctoring_service.get_proctoring_by_id(
            user=user, proctoring_id=proctoring_id
        )
    )
```

В данном примере рассмотрен маршрут, который должен вернуть информацию об определенной сессии прокторинга по ее `id`. В качестве

параметров принимаются `proctoring_id`, `proctoring_service` и `user`. `proctoring_id` – идентификатор сессии прокторинга, `proctoring_service` – сервис прокторинга с бизнес логикой, его клиент не передает, он передается в функцию как зависимость, `user` – токен пользователя, который передается клиентом и благодаря инструменту зависимостей дешифруется в объект пользователя. Далее все эти данные попадают в метод сервиса прокторинга, в котором заложена основная бизнес логика.

Сервисная логика реализована в модуле `services/proctoring/service.py`. Здесь происходит проверка прав пользователя, обработка ошибок и извлечение данных из базы. Ниже показан фрагмент метода:

```
@check_read_rights
async def get_proctoring_by_id(
    self, *, proctoring_id: int
) -> ProctoringItemSchema:
    return await (
        self.get_proctoring_by_id_if_exist(
            proctoring_id=proctoring_id
        )
    )
```

Метод `get_proctoring_by_id_if_exist` возвращает данные о сессии прокторинга, если такая сессия существует в базе данных. Внутри него происходит обращение к сервису базы данных, в котором, непосредственно, происходит запрос в базу данных.

Все взаимодействие происходит через сервис базы данных, все сервисы базы данных наследуются от основного сервиса, в котором прописана логика подключения к базе и создание асинхронных сессий. Все это происходит благодаря использованию библиотеки `SQLAlchemy`. Благодаря такому подходу получается переиспользовать уже написанный код и соблюдать принцип DRY [42].

Вся информация об API автоматически собирается в документацию Swagger, благодаря OpenAPI-интеграции с FastAPI. Такая интеграция позволяет удобно тестировать работоспособность приложения, без использования сторонних программ, проверять структуру запроса, структуру ответа, а также поддерживает единое описание интерфейса в актуальном состоянии.

Можно выделить следующие основные группы маршрутов:

- авторизация: `/api/v1/authorization`;
- пользователи и роли: `/api/v1/user`, `/api/v1/role`;
- предметы: `/api/v1/subject`;
- сессии прокторинга: `/api/v1/proctoring`;
- типы и результаты прокторинга: `/api/v1/proctoring-type`, `/api/v1/proctoring-result`.

Каждый маршрут требует авторизации через JWT и выполняет проверку прав доступа в зависимости от роли пользователя (`admin`, преподаватель, студент и тд.). Это реализовано через вспомогательные зависимости и декораторы (`decode_user_token`, `check_read_rights` и другие).

Таким образом, REST API приложения реализован в строгом соответствии с принципами модульности, изоляции слоёв и безопасности. Такое решение обеспечивает масштабируемость, простоту сопровождения и возможность расширения функционала без изменений архитектуры.

3.3 Интеграция с модулем анализа изображений

Основной функцией приложения прокторинга является фиксация нарушений, которые совершают студенты, во время проверки знаний. Для реализации данной функции используется отдельный модуль, анализа изображений, основанный на алгоритмах компьютерного зрения. Этот модуль

был разработан сотрудником ВУЗа, в рамках работы мы проводили интеграцию и взаимодействие с данным модулем.

Модуль анализа реализован в каталоге `services/mediapipe` и включает следующие компоненты:

- `face_landmark.py`, `head_pose_tracking.py`, `eyes_tracking.py`, `mouth_tracking.py` — трекинг лица, глаз, головы и рта;
- `speech_rec.py`, `people_counter.py` — фиксация речи и количества лиц в кадре;
- `start_proctoring.py` — основной модуль, отвечающий за запуск анализа;
- `utils.py` — вспомогательные функции.

Логика работы с алгоритмом достаточно простая, она схожа с работой обычных маршрутов в системе. Был создан отдельный маршрут для проверки изображений, это POST-запрос на эндпоинт `/api/v1/proctoring/check`. На входе приложение получает изображение и информацию о сессии прокторинга, которой принадлежит это изображение. После получения изображения необходимо проверить, есть ли такая сессия прокторинга или нет, узнать тип прокторинга, чтобы понять, какие данные необходимо отследить, далее изображение передается в модуль `mediapipe`. После получения результатов они записываются в таблицу `proctoring_result`.

На рисунке 3.3 представлена диаграмма последовательности, иллюстрирующая взаимодействие компонентов на уровне сервера:

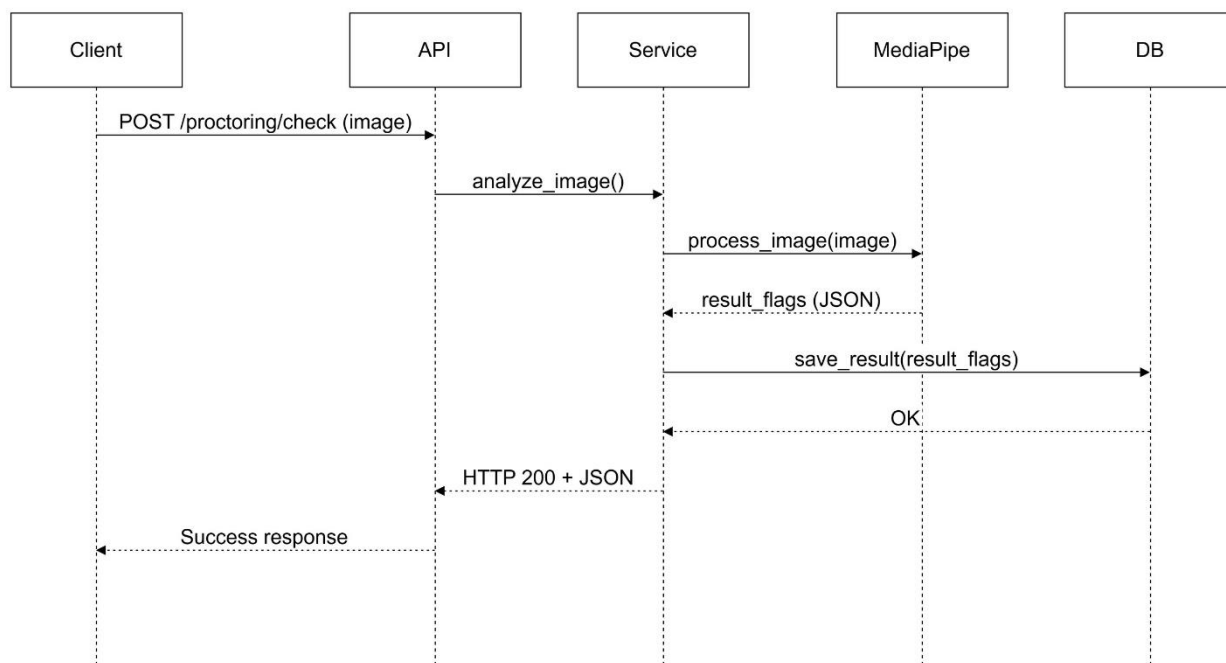


Рисунок 3.3 – маршрут проверки изображения на наличие нарушений

Результатом работы алгоритма является JSON-объект, содержащий булевы признаки по каждому контролируемому параметру, например:

- `detected_absence_person`: пользователь отсутствует в кадре;
- `detected_extra_person`: зафиксирован посторонний человек;
- `detected_person_substitution`: выявлена подмена личности;
- `detected_looking_away`: пользователь отворачивается;
- `detected_mouth_opening`: открыт рот (возможен шёпот);
- `detected_hints_outside`: зафиксированы внешние подсказки.

Эти значения сохраняются в базу данных, в таблицу `proctoring_result` и используются для генерации отчетов и предоставления необходимой информации пользователям системы. Структура таблицы спроектирована с учетом возможного добавления новых параметров анализа, что позволяет масштабировать систему без изменения архитектуры.

Благодаря модульному подходу удалось добиться четкого разделения ответственности между модулями. Сервис не содержит логику обработки

изображений, в нем записана только бизнес-логика. Это позволяет существенно упростить отладку и ускорить разработку приложения, позволяет легко заменять алгоритмы анализа изображения и добавлять новые параметры для анализа.

3.4 Хранение и отображение результатов прокторинга

Результаты прокторинга играют ключевую роль, они позволяют преподавателям понять, были ли зафиксированы нарушения и засчитывать ли прохождение контроля знаний. Как уже было сказано результаты прокторинга хранятся в отдельной таблице `proctoring_result`, связанной с таблицей сессий прокторинга `proctoring`, по внешнему ключу. Такое разделение обеспечивает гибкость и масштабируемость, поскольку в будущем возможно добавление новых признаков или параметров без необходимости переделывать основную модель сессии прокторинга.

Каждая запись в таблице `proctoring_result` содержит следующие булевы поля:

- `detected_absence_person` — зафиксировано отсутствие пользователя в кадре;
- `detected_extra_person` — обнаружено наличие постороннего лица;
- `detected_person_substitution` — выявлена подмена личности;
- `detected_looking_away` — пользователь отворачивался от экрана;
- `detected_mouth_opening` — открытие рта, возможный шёпот;
- `detected_hints_outside` — признаки получения внешних подсказок.

Это основные поля, который отвечают за результат пройденной сессии прокторинга. Дополнительно каждая запись содержит поля `created_at` и

updated_at, которые позволяют понять, когда был создан результат прохождения прокторинга и когда изменен в последний раз. Сохранение результатов происходит на уровне сервисного слоя после получения результатов от модуля анализа изображений. Вызов соответствующего метода фиксирует значения в таблице результатов и ассоциирует их с сессией прокторинга.

Для доступа к результатам в системе реализованы маршруты:

- GET /api/v1/proctoring-result — получить список всех результатов (доступно преподавателю по его предметам);
- GET /api/v1/proctoring-result/{id} — получить подробную информацию по конкретной сессии;
- PATCH /api/v1/proctoring-result/{id} — (при необходимости) обновить результат вручную, если был выявлен сбой в автоматическом анализе.

Доступ к результатам прокторинга реализован с учетом прав пользователей, преподаватели могут просматривать и изменять только результаты назначенных им предметов. Проверка прав доступа осуществляется на уровне сервисов, при помощи встроенных зависимостей FastAPI и самописных декораторов.

Таким образом, хранение и отображение результатов прокторинга реализованы в виде расширяемого, изолированного и безопасного механизма, обеспечивающего достоверность аналитики и удобство доступа к данным преподавателями.

3.5 Безопасность и контроль доступа

Одним из ключевых требований к информационным системам, особенно к системам, работающим с персональными данными и фотографиям

пользователей, является обеспечение безопасного доступа к данным и контроль действий пользователя в системе. В рамках реализации серверной части приложения особое внимание было уделено вопросам авторизации, аутентификации и разграничения прав доступа пользователей. Система построена по модели ролевого доступа, это означает, что у каждого пользователя есть своя роль, у роли есть набор прав, которые позволяют понять, к чему у пользователя должен быть доступ.

Базовые роли включают:

- Администратор, обладающий правами на управление пользователями, ролями и дисциплинами;
- Преподаватель, который может создавать сессии прокторинга, просматривать результаты, управлять типами прокторинга и предметами;
- Другие роли, например, «Студент» или «Техподдержка».

Каждому пользователю при регистрации назначается роль. Механизм прав доступа реализован через таблицу `role`, в которой указываются определенные роли с правами. Права обозначаются булевыми значениями. `rights_create`, `rights_read`, `rights_update`, `rights_delete` — позволяют чётко задать, какие типы операций разрешены данной роли.

Аутентификация пользователей реализована при помощи JWT (JSON Web Token). При успешной авторизации для пользователя генерируется токен, токен содержит данные о пользователе. Этот токен будет использоваться клиентами, клиенты передают этот токен в заголовке `Authorization` при каждом запросе к API. В каждом маршруте используется механизм зависимостей `FastAPI (Depends)`, через который происходит декодировка токена и получение информации о пользователе. Далее при обращении к сервису происходит проверка прав доступа через самописные декораторы, такой подход

позволяет разграничить проверку прав и бизнес-логику приложения. Для добавления новой проверки необходимо будет изменить уже существующий декоратор или создать новый и применить его в методе сервиса.

Особую роль играет связка преподавателя с предметами, необходимо отображать только те предметы, которые привязаны к преподавателю и не давать доступ к другим предметам. Эта логика реализована на уровне фильтрации данных в сервисе и контролируется во время выполнения запроса.

Для авторизации пользователю необходим логин и пароль. Для защиты паролей используется механизм хэширования. При регистрации или изменения данных о пользователе в базу данных сохраняется только хэш пароля. Прямой доступ к базе данных отсутствует, все взаимодействие с ней происходит исключительно через проверенный API-интерфейсы.

Таким образом, реализованная система безопасности обеспечивает:

- достоверную аутентификацию и идентификацию пользователей;
- чёткое разграничение доступа к функциям и данным;
- защиту передаваемой информации и целостность данных.

Все перечисленные механизмы соответствуют базовым принципам безопасной разработки и могут быть легко расширены при необходимости: например, добавлением двухфакторной аутентификации или журналированием действий пользователей.

ЗАКЛЮЧЕНИЕ

В ходе выполнения магистерской диссертационной работы была разработана серверная часть интеллектуального веб-приложения прокторинга, предназначенного для использования в системе дистанционного контроля знаний в высшем образовании. Целью проекта являлось создание устойчивой, расширяемой и безопасной платформы, которая бы позволяла преподавателям назначать экзамены, контролировать прохождение сессий прокторинга и получать автоматизированные отчёты о возможных нарушениях.

На этапе проектирования была выбрана архитектура монолитного REST API с чётким разделением по слоям: роутеры, сервисы, слой доступа к данным и база данных. Разработка велась с использованием современных технологий: FastAPI, SQLAlchemy, PostgreSQL, Alembic, Pydantic, JWT. Структура проекта построена таким образом, чтобы обеспечить простоту масштабирования и сопровождения.

Реализация охватывает все ключевые аспекты серверной логики: регистрацию и авторизацию пользователей, управление ролями и правами доступа, создание и настройку сессий прокторинга, обработку изображений, взаимодействие с модулем компьютерного зрения, а также хранение и отображение результатов. Особое внимание было уделено безопасности: реализована система ролевого доступа и защита данных на основе токенов.

Алгоритмы анализа изображений, фиксирующие потенциальные нарушения (отсутствие пользователя в кадре, присутствие посторонних лиц, отклонение взгляда и пр.), были реализованы другим сотрудником университета. В рамках данной работы была выполнена интеграция этих алгоритмов с веб-приложением, что позволило связать результаты анализа с конкретными сессиями прокторинга и преподавателями.

Все цели, поставленные в начале работы, были достигнуты. Разработанный прототип прошёл ручное тестирование, продемонстрировал стабильную работу и соответствует требованиям, предъявляемым к подобным системам в образовательной среде. Полученные результаты могут быть использованы в учебных заведениях для повышения прозрачности и объективности контроля знаний при проведении дистанционного обучения.

В дальнейшем возможно расширение системы путём внедрения автоматической генерации отчётов, поддержки видеоаналитики в реальном времени, интеграции с LMS-платформами и расширения функциональности интерфейса преподавателя.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Современные проблемы науки и образования. 2024. № 6 URL: <https://science-education.ru/ru/article/view?id=33752> (дата обращения: 03.04.2025)
2. Антонова Н.Л., Абрамова С.Б. Дистанционное образование: онлайн-прокторинг как инструмент оценки результатов обучения. Университетское управление: практика и анализ. 2023;27(2):5-16.
3. Абакумов А. А ПРОРЫВЫ В ОНЛАЙН-ПРОКТОРИНГЕ: РАЗРАБОТКА И ВНЕДРЕНИЕ СИСТЕМ НА БАЗЕ ИИ // Universum: технические науки. 2024. №5 (122). URL: <https://cyberleninka.ru/article/n/proryvy-v-onlayn-proktoringe-razrabotka-i-vnedrenie-sistem-na-baze-ii> (дата обращения: 08.04.2025).
4. Examus - официальный сайт [Электронный ресурс]. URL: <https://ru.examus.net/> (дата обращения: 11.04.2025)
5. РЭУ Про – официальный сайт [Электронный ресурс]. – URL: <https://rea.proctoring.online/> (дата обращения: 11.04.2025).
6. ProctorEdu – официальный сайт [Электронный ресурс]. URL: <https://proctoredu.ru/> (дата обращения: 11.04.2025)
7. Stepik – официальный сайт [Электронный ресурс]. URL: <https://stepik.org> (дата обращения: 11.04.2025)
8. Stepik помогает обеспечивать качество обучения в онлайн-магистратуре МФТИ [Электронный ресурс]. URL: https://old.mipt.ru/education/departments/fpmi/news/stepik_pomogaet_obespechivat_kachestvo_obucheniya_v_onlayn_magistrature_mfti (дата обращения: 11.04.2025)
9. Цыгалов Ю. М. Эффекты и риски дистанционного образования в высшей школе // Управленческое консультирование. 2020. № 10. С. 61–73.
10. Мухамадиева К. Б. КОМПЬЮТЕРНОЕ ЗРЕНИЕ ДЛЯ АНАЛИЗА ПРОЦЕССА ОБУЧЕНИЯ // Экономика и социум. 2025. №2-1 (129). URL:

<https://cyberleninka.ru/article/n/kompyuternoe-zrenie-dlya-analiza-protsessa-obucheniya> (дата обращения: 24.04.2025).

11. Face Recognition using Python [Электронный ресурс]. – URL: https://github.com/ageitgey/face_recognition (дата обращения: 25.04.2025).

12. dlib: Toolkit for Machine Learning and Computer Vision [Электронный ресурс] – URL: <http://dlib.net> (дата обращения: 25.04.2025).

13. Histogram of Oriented Gradients [Электронный ресурс] – URL: https://scikit-image.org/docs/stable/auto_examples/features_detection/plot_hog.html#id3 (дата обращения: 25.04.2025).

14. Ghosh, Anirudha & Sufian, A. & Sultana, Farhana & Chakrabarti, Amlan & De, Debashis. (2020). Fundamental Concepts of Convolutional Neural Network. 10.1007/978-3-030-32644-9_36.

15. LFW Dataset [Электронный ресурс] – URL: <https://datasets.activeloop.ai/docs/ml/datasets/lfw-dataset/> (дата обращения: 25.04.2025).

16. VGGFace2 Dataset for Face Recognition [Электронный ресурс] – URL: https://github.com/ox-vgg/vgg_face2?tab=readme-ov-file (дата обращения: 25.04.2025).

17. MS-Celeb-1M: A Dataset and Benchmark for Large-Scale Face Recognition [Электронный ресурс] – URL: <https://www.microsoft.com/en-us/research/publication/ms-celeb-1m-dataset-benchmark-large-scale-face-recognition-2/> (дата обращения: 25.04.2025).

18. Li, Haotian & Xu, Min & Wang, Yong & Wei, Huan & Qu, Huamin. (2021). A Visual Analytics Approach to Facilitate the Proctoring of Online Exams. 10.48550/arXiv.2101.07990.

19. MediaPipe Face Mesh: Real-Time Face Tracking [Электронный ресурс]. – URL: https://google.github.io/mediapipe/solutions/face_mesh (дата обращения: 03.05.2025)

20. Object Detection in OpenCV [Электронный ресурс]- URL:<https://www.opencvhelp.org/tutorials/image-analysis/object-detection/> (дата обращения: 03.05.2025)

21. SSD: Single Shot MultiBox Detector [Электронный ресурс] - URL: https://link.springer.com/chapter/10.1007/978-3-319-46448-0_2 (дата обращения: 05.05.2025).

22. You Only Look Once: Unified, Real-Time Object Detection [Электронный ресурс] - URL: <https://arxiv.org/abs/1506.02640> (дата обращения: 05.05.2025).

23. Букина Т.В. Искусственный интеллект в образовании: современное состояние и перспективы развития // Общество: социология, психология, педагогика. 2025. №1. URL: <https://cyberleninka.ru/article/n/iskusstvennyy-intellekt-v-obrazovanii-sovremennoe-sostoyanie-i-perspektivy-razvitiya> (дата обращения: 07.05.2025).

24. Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных» [Электронный ресурс]. – URL: https://www.consultant.ru/document/cons_doc_LAW_61798/ (дата обращения: 07.05.2025)

25. General Data Protection Regulation [Электронный ресурс] - URL: <https://gdpr-info.eu/> (дата обращения: 10.05.2025)

26. Барбосов А.Н., Богатов А.А., Давыдов С.С., Кокорин Е.В., Шаров И.Н. ПРОБЛЕМЫ ОБЕСПЕЧЕНИЯ КАЧЕСТВА ОНЛАЙН-ОБУЧЕНИЯ В ОБРАЗОВАТЕЛЬНЫХ ОРГАНИЗАЦИЯХ ВЫСШЕГО ОБРАЗОВАНИЯ // Современное педагогическое образование. 2022. №4. URL: <https://cyberleninka.ru/article/n/problemy-obespecheniya-kachestva-onlayn-obucheniya-v-obrazovatelnyh-organizatsiyah-vysshego-obrazovaniya> (дата обращения: 10.05.2025).

27. REST API Design Rulebook / M. Masse. – O'Reilly Media, 2011. – ISBN: 978-1-4493-3162-0.

28. Docker - официальный сайт [Электронный ресурс] - URL: <https://www.docker.com/> (дата обращения: 13.05.2025)
29. JWT Authorization [Электронный ресурс] - URL: <https://jwt.io/> (дата обращения: 13.05.2025)
30. The OpenAPI Specification [Электронный ресурс]. – URL: <https://learn.openapis.org/specification/> (дата обращения: 13.05.2025).
31. FastAPI Documentation [Электронный ресурс]. – URL: <https://fastapi.tiangolo.com/> (дата обращения: 15.05.2025).
32. PostgreSQL Documentation [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/> (дата обращения: 15.05.2025).
33. SQLAlchemy Documentation [Электронный ресурс]. – URL: <https://docs.sqlalchemy.org/> (дата обращения: 15.05.2025).
34. Alembic Documentation [Электронный ресурс]. – URL: <https://alembic.sqlalchemy.org/> (дата обращения: 15.05.2025).
35. Pydantic Documentation [Электронный ресурс]. – URL: <https://docs.pydantic.dev/> (дата обращения: 15.05.2025).
36. Слоистая архитектура приложений: как обеспечить поддерживаемость доменного слоя [Электронный ресурс]. – URL: <https://habr.com/ru/articles/802617/> (дата обращения: 01.06.2025).
37. Полларда, Б. HTTP/2 в действии / Б. Полларда // ДМК-Пресс. – 2021. – с. 25 – 60.
38. MediaPipe: Google Machine Learning Framework [Электронный ресурс]. – URL: <https://mediapipe.dev/> (дата обращения: 15.05.2025).
39. OpenCV: Open Source Computer Vision Library [Электронный ресурс]. – URL: <https://opencv.org/> (дата обращения: 15.05.2025).
40. Комаров, В. И. Путеводитель по базам данных / В. И. Комаров // ДМК-Пресс. – 2024. – с. 33 – 37.
41. Требования ACID на простом языке [Электронный ресурс]. – URL: <https://habr.com/ru/articles/555920/> (дата обращения: 01.06.2025)

42. Мартин, Р. Чистый код: создание, анализ и рефакторинг / Р. Мартин
// Питер. – 2022 – с. 73 – 74.

ПРИЛОЖЕНИЕ

Программный код расположен по адресу:

<https://github.com/Ygorik/proctoring>

Выпускная квалификационная работа выполнена мной совершенно самостоятельно. Все использованные в работе материалы и концепции из опубликованной научной литературы и других источников имеют ссылки на них.

«20» июня 2025 г.

И^о7Е3
(подпись)

Павлов Е.И.
(Ф.И.О)